

A System of Refusals

The SARSI Agents, Built

An Implementation Report: One Loop, Nineteen Planes, the Plan Made
Between Worker and Session, and the Questions an Agent May Not Answer

Chengshuai Yang

NextGen PlatformAI C Corp.

`spiritai@platformai.org`

August 3, 2026

Abstract

Three prior papers designed a console: five agents behind one chat box, the same console once an account owns several machines, and the worker layer between a person's request and a session on their hardware. This paper reports the system actually built from them on a single host — nineteen modules, one loop — and it differs from all three in a way worth stating first: **almost every module in it exists to refuse something**. The affirmative behaviour in each case is short and unsurprising. The refusals are where the design lives, and they are what took the effort to get right.

Two decisions in the loop could reasonably have gone the other way. A message does *not* start a session: it produces a task and a plan and stops, because a message that silently spawned a governed session would make “I asked a question” and “I authorised work” the same gesture. And the manager routes by *creating a task on a worker* rather than by driving a session — a prohibition enforced where execution happens, since the assignment call raises, rather than remembered in prose.

Three results emerged from building rather than from designing. **The plan is made between the worker and the session, through a file.** A plan drafted by one model in one shot has never seen the repository it describes; a plan drafted by the session alone is the session writing the criteria it will then be judged against. So the worker seeds a draft from its own workspace, a planning session capped below the work ceiling improves it in place, the worker checks it against the rules the verifier will depend on, and only then is it promoted. The exchange happens through a path on disk rather than a terminal, because every state-reading bug this design inherited came from reconstructing something off a screen that redraws.

A session's questions divide into two kinds that look identical and are opposite. “Which directory should I index?” wants a fact the worker holds; “Do you want to proceed?” asks whether the session may act. Conflating them is how an agent approves its own actions because it knew what the owner would want. The permission check therefore runs first and declines outright — the model is never shown a gate — then a reserved class is refused however it is phrased, then the worker answers only from grounded material and otherwise says it does not know. Every outcome records the evidential rung it stands on.

And precedence is mechanical. Entering interact mode pauses the worker's steering, because otherwise the worker types its next step into a terminal the owner is using and the two contend for the prompt.

Part II specifies all nineteen planes: what each holds, what it refuses, its interface, its characteristic failure, and its deployment status. A closing section lists what remains unbuilt.

Keywords: SARSI; multi-agent systems; agent governance; authority; implementation report; refusal as design; planning; system design.

Contents

1	Introduction	6
1.1	Three designs, one implementation	6
1.2	The observation that organises everything	6
1.3	What “built” means here	6
2	The Loop	6
2.1	Nine steps	6
2.2	Two decisions that could have gone the other way	7
2.3	Routing without guessing	7
3	What the Predecessors Specified, and What Exists	7
3.1	The pattern in the delta	7
4	The Evidence Ladder, as Implemented	8
5	The Workspace, and Why the Planner Needed One	9
5.1	w_name: the node’s workspace is its history	9
5.2	The defect this closed, and its twin one level down	9
6	Ledgers: Four Files, Not One	9
7	Registry — config.py (262 lines)	11
7.1	Role	11
7.2	Refuses	11
7.3	Interface	11
7.4	Deployment status	11
8	Router — router.py (126 lines)	11
8.1	Role	11
8.2	Refuses	12
8.3	Characteristic failure	12
8.4	Deployment status	12
9	Gateway and Transports — gateway.py (113 lines)	12
9.1	Role	12
9.2	Refuses	12
9.3	Deployment status	12
10	Conversation — conversation.py (123 lines)	12
10.1	Role	12
10.2	What it holds	12
10.3	Characteristic failure	13
10.4	Deployment status	13

11 Manager — <code>loop.py</code> (139 lines)	13
11.1 Role	13
11.2 Refuses	13
11.3 Characteristic failure	13
11.4 Deployment status	13
12 Worker Plane — <code>plane.py</code> (191 lines)	13
12.1 Role	13
12.2 Refuses	13
12.3 Characteristic failure	14
12.4 Deployment status	14
13 Tasks — <code>task.py</code> (343 lines)	14
13.1 Role	14
13.2 States, and the rule they follow	14
13.3 What it holds	14
13.4 Interface	14
13.5 Refuses	15
13.6 Deployment status	15
14 Planning — <code>planner.py</code> (128) and <code>planning.py</code> (378 lines)	15
14.1 The middle that both extremes are worse than	15
14.2 Three things this is careful about	15
14.3 Interface	16
14.4 Refuses	16
14.5 Characteristic failure	16
14.6 Deployment status	16
15 Session — <code>session.py</code> (397 lines)	16
15.1 Role	16
15.2 Two rules, enforced rather than described	16
15.3 Interface	17
15.4 Refuses	17
15.5 Characteristic failure	17
15.6 Deployment status	17
16 Answering — <code>answer.py</code> (165 lines)	17
16.1 The two stops	17
16.2 The order is the mechanism	17
16.3 Deployment status	18
17 Vault — <code>vault.py</code> (197 lines)	18
17.1 Role	18
17.2 Two stages	18
17.3 Refuses	18
17.4 Deployment status	18

18 Outward Acts — <code>outward.py</code> (218 lines)	18
18.1 Role	18
18.2 Deployment status	19
19 Verification — <code>verify.py</code> (192 lines)	19
19.1 Role	19
19.2 Refuses	19
19.3 Characteristic failure	19
19.4 Deployment status	19
20 Modes — <code>modes.py</code> (216 lines)	19
20.1 The three modes	19
20.2 Precedence is mechanical	20
20.3 Deployment status	20
21 Board and Commands — <code>board.py</code> (116), <code>commands.py</code> (162 lines)	20
21.1 Role	20
21.2 Refuses	20
21.3 What it shows	20
21.4 Deployment status	20
22 The Console Above It	21
23 A Request, Traced	21
23.1 The request	21
23.2 RT — routing	21
23.3 TSK — a task, not a session	21
23.4 PLN — the plan is made between two parties	22
23.5 GRT — the permissions the plan declared	22
23.6 ASG — the owner opts in	22
23.7 CC — and a question	22
23.8 VER — and REP	23
23.9 Where the owner was involved	23
24 Below ASG: The Session Plane	23
24.1 Verification is an ordering rule, not a step	23
24.2 Reporting is not ruling	23
24.3 The stuck detector separates four situations	23
25 The Seam to the Console, in Full	23
25.1 One module, plain data	24
25.2 Why the surface is read-only	24
25.3 Two absences that are the design	24
25.4 The failure this surface had to survive	24
26 Failure Modes This System Adds	24

27 A Second Trace: When It Reaches the Owner	24
27.1 No machine can do it	25
27.2 The plan declares a permission that is never granted	25
27.3 The vault denies	25
27.4 The session asks something nobody can ground	25
27.5 The verifier fails the work	26
27.6 The owner takes the wheel mid-flight	26
28 A Taxonomy of the Refusals	26
28.1 Authority refusals	26
28.2 Evidence refusals	26
28.3 Honesty refusals	26
28.4 Blast-radius refusals	27
29 How This Was Verified	27
29.1 Tests that passed for the wrong reason	27
29.2 Tests that touched the live host	27
29.3 What the source corrected in the design	28
30 Two Sessions, One System	28
30.1 What did not go wrong	28
30.2 What did	28
31 What Is Not Built	29
32 What the Owner Is Asked, and How Often	29
32.1 The complete list	29
32.2 Which are load-bearing and which are debt	30
32.3 Why this framing matters for autonomy claims	30
33 Limitations, and What This Report Cannot Claim	30
33.1 Refusals are verified structurally, not adversarially	30
33.2 Scale is one owner, one host, and a short history	30
33.3 The self-model is thinner than its papers	31
33.4 Two of the four channels are absent	31
33.5 The report was written by a participant	31
34 Conclusion	31
A Index of Refusals	31
B Deployment Status by Plane	32

PART I — THE SUBSTRATE

1 Introduction

1.1 Three designs, one implementation

This is an implementation report. Its predecessors specify a console fronted by a manager with five agents behind it [2], the same console once an account owns several machines [1], and the worker layer standing between a person’s request and a session on their machine [6]. The underlying self-model architecture is given separately [3, 5] as is the fleet coordination semantics [4].

What follows is the system built from those on one host, and — more usefully — the places where building it changed the design. A design paper argues that a property is desirable. An implementation report can say which properties survived, and that turns out to be a much smaller and more specific set.

1.2 The observation that organises everything

Most of this system is a catalogue of things that will not happen

Nineteen modules were implemented. Each is most economically described by what it *refuses*: a stranger’s message, a command to the manager, a plan without criteria, a control from a non-owning worker, a permission gate shown to a model, a secret leaving the machine, a verdict from the agent that produced the work, a session started without an owner’s opt-in.

In each case the affirmative behaviour is a handful of lines and holds no surprises. The refusal is the part that required thought, the part that has a failure story behind it, and the part that would be quietly lost in a rewrite.

This is not a stylistic observation, and it has a practical consequence for how such systems should be specified. A system described by what it permits accumulates capability and discovers its boundaries later, usually from an incident. A system described by what it refuses states the boundary first and lets capability grow inside it. The second is harder to write and much easier to review, because a reviewer can check a refusal by trying to violate it.

1.3 What “built” means here

Every claim in Part II is checked against the source, not against the design documents that preceded it. Where the design and the implementation disagree, the implementation is reported and the disagreement noted. Line counts are given per module so the reader can weigh how much of the system each section covers. Deployment status is stated per plane, and §31 lists what is designed and absent.

2 The Loop

2.1 Nine steps

$$U \rightarrow RT \rightarrow (MGR \mid AGT) \rightarrow TSK \rightarrow PLN \rightarrow [GRT] \rightarrow ASG \rightarrow CC \rightarrow VER \rightarrow REP$$

A message arrives (U) on either surface. The router (RT) decides which agent owns it and whether the sender may speak at all. Either the manager (MGR) or the addressed agent (AGT) takes it. A task is created (TSK); a plan is made (PLN); the permissions that plan declares are asked for (GRT). The worker hands the

plan — not the request — to a session (ASG); the session works (CC); an independent verifier rules (VER); a report returns (REP).

2.2 Two decisions that could have gone the other way

Rule 1 (A message does not start a session). *An inbound message produces a task and a plan, and stops. The owner reads the plan and says start.*

The alternative — a message that spawns a governed session directly — is more responsive, and collapses a distinction that must not collapse: it makes “*I asked a question*” and “*I authorised work*” the same gesture. The cost is one extra turn before anything runs. The benefit is that authorisation stays an act the owner performs rather than a side effect of conversing. The build ladder is owner-opt-in throughout, and this is where that ladder starts.

Rule 2 (The manager routes by creating a task, never by driving a session). *Creating a task on a worker is telling a worker to work, which the manager may do. Driving a session is executing, which it may not. `session.assign` raises when the manager attempts it.*

The second sentence carries the weight. A prohibition recorded only in documentation holds until someone adds a feature; a prohibition that raises at the point of execution is a property of the system. Throughout this report, the question asked of every rule is: *where does it raise?*

2.3 Routing without guessing

The manager sends work it has not been told a home for to a default worker whose stated purpose is “any task with no better home.” It does not infer a specialist from the sentence.

Design principle 1 (Vocabulary is not routing evidence). *Naming a worker in the message wins. Everything else goes to the base worker. Guessing a specialist from a sentence would put mail work in the funding agent on a coincidence of vocabulary, and the owner would have no way to see why.*

3 What the Predecessors Specified, and What Exists

Three design papers preceded this system. Reporting the delta is more useful than reporting the result, because the pattern in the delta is itself a finding.

3.1 The pattern in the delta

Three kinds of change appear, and they are not equally distributed.

What survived unchanged was invariably a refusal with a named enforcement point: the vault’s interface, the ban on self-verification, the manager’s inability to execute.

What was narrowed was invariably a mechanism specified by structure rather than by consequence — per-node salience weights, a four-channel taxonomy. These were not wrong; they were unfalsifiable at this scale, so the implementation built the part that had an observable effect and left the rest.

What changed outright was the plan’s authorship, and it changed because building it revealed a constraint no design document had noticed: a plan drafted by one party is either uninformed about the repository or written by the party it will judge. Neither failure is visible from a specification. Both are obvious the first time a plan names a file that does not exist.

Table 1: Design intent against implementation.

Specified	Built as	Delta
Five agents behind one chat box	An agent registry with a configurable roster	The number was never structural. What matters is the binding table and the owner lock, neither of which the design named
A manager that coordinates and does not execute	Same, with the prohibition raising at the assignment call	Survived, and strengthened: the design stated it, the implementation enforces it
Communicators as a server-side plane	Absent on one host	The plane exists to separate availability from execution across machines. On one host there is nothing to separate, and building it anyway would have been architecture for its own sake
Four inter-agent channels	Report and directive built; prior bias and pre-emption not	The two built are the two that carry work. Prior bias was designed as unable to move an estimate, so its absence changes no conclusion
A vault answering yes or no	Same	Survived unchanged — the only plane that did
Drafting is not sending	Same, plus byte equality at transmission	Strengthened: the design permitted an approval flow that re-renders before sending
A bounded workspace per node	One workspace per agent, feeding the planner	Narrowed: the design specified per-node admission policies with salience weights; what exists is one context per agent
The plan drafted before work begins	The plan made <i>between</i> worker and session	Changed outright, and this is the largest delta in the system (§14)

4 The Evidence Ladder, as Implemented

The architecture papers define a seven-rung ladder from signed governance metadata down to an agent’s own narration. The implementation touches five of them, and it is worth recording exactly where, because “we use an evidence ladder” is the kind of claim that decays into decoration.

L4 (audited task outcomes) is thin — outcomes are recorded but not yet aggregated into anything that reads them back. L7 is present only as the thing the system refuses to treat as evidence.

Rule 3 (An answer carries the rung it stands on). *Every answering outcome records whether the answer came from the owner (L5) or from the worker’s inference (L6).*

This is the smallest change in the implementation and among the most important. Storing an answer as merely *an answer* is indistinguishable from the correct behaviour in the happy case, and unrecoverable in the case that matters — the one where something went wrong and the question is who decided.

Table 2: Where each rung actually appears in the built system.

Rung	What it is	Where it appears here
L1	signed governance metadata	the registry file: which agents exist, which bindings are legal, who the owner is
L2	runtime instrumentation	the ledgers; task state on disk; the draft’s fingerprint; whether a tmux session exists
L3	independent evaluation	the verifier’s verdict — a separate judge, given the plan’s criteria and the session’s visible evidence
L5	owner declarations	every grant; every answer the owner gives; the owner-edited flag on a plan
L6	model inference	an answer the worker infers; a drafted plan before it is checked

5 The Workspace, and Why the Planner Needed One

5.1 `w_name`: the node’s workspace is its history

Each agent owns one directory: notes it wrote for itself, the conversation it has had with the owner across every surface, and what it learned from tasks it already completed. A sibling holds facts about the machine. Neither is inside the other.

Design principle 2 (An agent that plans without reading its own history plans the same task again). *It re-asks a question answered last week, names a path the owner corrected it on, and proposes a phase that is already done. The workspace is the only place that knowledge lives, so it must be read before a plan is written — not after, when it can only be regretted.*

Three readers are offered so callers can take what they can afford: the agent’s deliberate notes; a recall of what was actually said, most recent last; and a combined, labelled context small enough to put in a prompt.

5.2 The defect this closed, and its twin one level down

Declared and never fed

The planner’s drafting function accepted a context argument, and its docstring stated exactly why one was needed. Nothing passed it. The parameter existed; the workspace behind it did not.

This is the same defect the session-level design had already hit one level down, where a composer node declared four history sources and was fed none of them — and could therefore re-issue an instruction that had just failed. A declared-but-unfed input is worse than an absent one, because the declaration is read as a description of behaviour.

The remedy is the same at both levels: feed the node, bound the selection, and *count the overflow rather than truncating it silently*. An unreported truncation is a node quietly deciding what the owner does not get to see.

6 Ledgers: Four Files, Not One

Append-only ledgers record what happened, split by kind of event rather than merged into one stream.

Design principle 3 (A mixed log makes the question harder than the answer). *Four files, one per kind, because the questions asked of a ledger are of the form “what did the owner authorise” or “what left this machine”, and a single mixed log turns each of those into a filtering exercise over everything that ever happened.*

The separation also has an authority consequence: the record of what the owner authorised must not be writable by the ordinary path that records what an agent did, or the two become one claim.

PART II — THE NINETEEN PLANES

Each section gives the plane’s role, what it holds, what it refuses, its characteristic failure, and its deployment status. Line counts indicate weight.

7 Registry — `config.py` (262 lines)

7.1 Role

The agent registry, read from a single file. It is modelled on the shape of an existing multi-agent chat runtime the owner already runs on this host — an agent list, per-channel accounts, and bindings that say which agent owns which account.

Read against the real thing, not a description of it

Modelling the registry on the running system’s actual configuration file rather than on a specification of it corrected three things: the owner lock is an *allowlist* of ids rather than a single owner id (one id is an allowlist of length one, so the list is kept); a binding carries an explicit type; and an agent may name its own workspace and agent directory rather than having them derived. Each correction was invisible from the spec and obvious from the file.

7.2 Refuses

An agent not in the registry. A sender not on the owner allowlist. A binding that does not validate. Absence of the registry file is distinguished from a malformed one — the first means *not yet initialised* and is actionable, the second is a fault.

7.3 Interface

Registry

`load(path=None, *, validate=True) -> Registry` — reads the registry file. Raises a distinguished configuration error when the file is absent, which the console renders as *not yet initialised* rather than as a fault.

`Registry.agents` — a list, not a map. Lookup is by identifier through the registry rather than by dictionary access, so an unknown agent is one code path and not two.

Agent — carries `role`, its workspace and agent directories, and its bindings. There is no `kind` field; a consumer that assumes one silently reads empty.

7.4 Deployment status

Built. The distinction between *not initialised* and *broken* is surfaced to the console as an actionable message rather than a server error.

8 Router — `router.py` (126 lines)

8.1 Role

One inbound message to one agent. Two doors — a chat transport and a command line — pass through the same routing so that neither acquires a rule the other lacks.

8.2 Refuses

Rule 4 (A stranger is dropped, counted, and never answered). *Not even a refusal is returned. A reply would confirm that the agent exists and is listening.*

Rule 5 (A dropped turn is counted, not transcribed). *An unknown sender must not be able to write arbitrary text into the owner's records. The count is the evidence; the content is not retained.*

The second rule is easy to miss and does real work: a system that logs the text of messages it refuses has given an unauthenticated party a writable channel into the owner's history.

8.3 Characteristic failure

Both doors drifting apart — a rule added to the chat path and not the command path. Routing both surfaces through one module is the structural answer.

8.4 Deployment status

Built, both surfaces.

9 Gateway and Transports — `gateway.py` (113 lines)

9.1 Role

One daemon hosting every agent's loop, with the transport injected so the rules above are testable without a network. A single entry point takes an inbound message from either surface and runs it through the loop.

9.2 Refuses

One agent's failure becoming the fleet's. A bot that errors, or an agent handler that raises, costs that turn and nothing else; the other agents keep running.

9.3 Deployment status

Built. The transport is injected, so the router's refusals are exercised in tests without network access — which is what makes them regression-proof rather than aspirational.

10 Conversation — `conversation.py` (123 lines)

10.1 Role

One agent, one conversation, whichever door the owner used. A plan made at the command line is visible and steerable in that agent's chat, and the reverse.

Rule 6 (The agent never re-asks something answered on the other surface). *An open question is keyed, and it closes from whichever surface answers it.*

10.2 What it holds

The full exchange with provenance for each entry: who said it, on which surface, and — for questions — whether they are open, answered, and by whom.

10.3 Characteristic failure

Two conversations diverging, so that the owner answers on one surface and is asked again on the other. The keyed-question mechanism exists specifically to prevent it.

10.4 Deployment status

Built, and consumed by the answering path (§16).

11 Manager — `loop.py` (139 lines)

11.1 Role

The connective tissue: a message becomes a task with a plan. Each plane below existed and was tested separately; this is what calls the next.

11.2 Refuses

Executing. The manager routes and answers. It never reaches the assignment step, and the assignment call raises if it tries (Rule 2).

11.3 Characteristic failure

The convenient shortcut: a manager that “just this once” drives a session because it is faster than routing. The raise is what makes it not a matter of discipline.

11.4 Deployment status

Built.

12 Worker Plane — `plane.py` (191 lines)

12.1 Role

A directive goes down from the agent the owner spoke to, to a worker; a report comes back up. Between them sit two questions that must be answered before anything is queued: whether this worker admits the directive, and whether this machine can do it.

12.2 Refuses

Rule 7 (A worker is not commanded). *A directive is evidence that something is wanted. The worker decides whether it can be done here.*

Rule 8 (Nothing waits quietly). *A directive that cannot be placed is refused and reported in the same breath. An unplaceable directive that waits is indistinguishable, from the chat, from work in progress.*

Rule 9 (A report may not claim success the verifier did not grant). *Marking a report verified without a verdict raises.*

12.3 Characteristic failure

Silent queueing — the failure Rule 8 exists for. It is attractive because it makes the system look tolerant of missing capability, and it converts a clear “no machine can do this” into an indefinite wait the owner cannot distinguish from progress.

12.4 Deployment status

Built, including the report path.

13 Tasks — `task.py` (343 lines)

13.1 Role

A worker holds several tasks. Each owns a plan file whose phases carry the criteria the verifier will judge.

13.2 States, and the rule they follow

Table 3: Task states. Each says something true; none of them is a euphemism.

State	Means
drafting	a plan is being made
awaiting-grant	the plan declared a permission the owner has not given
ready	granted, and waiting for the owner to say start
waiting	over the worker’s concurrency limit — and it says so rather than appearing idle
running	a session is working
awaiting-owner	a question is open that only the owner may answer
done / failed / refused	terminal

Design principle 4 (A state must not flatter). *Waiting is distinguished from ready because being over a limit is not the same as being about to start. Awaiting-owner is distinguished from running because a board that shows a blocked task as busy tells the owner the opposite of what is true.*

13.3 What it holds

Identity, state and reason; the plan path and its phases; declared permissions and the grants against them; whether the owner edited the plan; and whether a successor proposal is pending.

13.4 Interface

Tasks

`list_for(agent) -> [Task] · get(agent, name_or_id) -> Task | None` — no configuration argument; the agent carries what is needed.

Task is a property wrapper over a stored record, not a frozen structure: `id, name, state, reason, dir, plan_path, plan_text(), phases(), permissions(), grants, owner_edited, pending_proposal.`

`phases()` parses each heading with its criterion line, in order. It can raise on a malformed plan file — so a caller listing many tasks must guard per task, or one bad file empties the whole list.

`write_plan.grant.start.finish.polish` — the mutating surface, and the one a read-only consumer must not touch.

13.5 Refuses

Losing the owner's edit. Once the owner has rewritten a plan, a later polish becomes a *proposal* beside it rather than a replacement.

13.6 Deployment status

Built.

14 Planning — `planner.py` (128) and `planning.py` (378 lines)

This is the largest plane and the one that changed most between design and implementation.

14.1 The middle that both extremes are worse than

Two plausible designs, both wrong

The worker drafts the plan alone. A plan drafted by one model in one shot has never seen the repository it describes. It names files that do not exist and phases that were done last week.

The session drafts the plan alone. Then the session is writing the success criteria the verifier will later judge it against — and the verdict means nothing, because the standard and the work have the same author.

The plan is made between them

The worker writes the *first* plan, from the owner's words and its own workspace. That seed becomes a draft file on the task. A governed planning session opens *on that file*; the session reads the actual code and improves the plan in place. The worker reads it back, checks it against the rules the verifier will depend on, and guides another round for whatever is still wrong. When it passes, the worker promotes the draft to the plan.

14.2 Three things this is careful about

Rule 10 (The exchange happens through a file, not a screen). *Both parties read and write a path on disk, and the worker checks what is actually there.*

The justification is empirical rather than aesthetic: *every state-reading bug this design inherited came from reconstructing something off a terminal that redraws*. A file has a fingerprint, can be diffed, and does not scroll.

Rule 11 (A planning session is not a work session). *It is capped below the work ceiling, told in its brief to plan and not to do, and recorded separately.*

So the assignment step still means what it meant — the owner opting in to the work — and the refusal to start without a plan is untouched.

Rule 12 (Rounds are bounded). *A fixed number of improvement rounds, each with a timeout. Convergence is not assumed.*

14.3 Interface

Planning

`draft(request, *, task_name, drafter=None, context="") -> str` — the seed. context is the agent’s workspace, and the reason it exists is Principle 2.

`check(text) -> [problem]` — the rules the verifier will depend on, evaluated before anything is promoted. Problems are returned, never repaired.

`draft_path(task) · read_draft(task)` — the file the exchange happens through.

`open_session · guide · wait_for_draft · close` — the round loop, bounded by a round count and a per-round timeout.

A fingerprint over the draft answers *did this actually change* without reading a screen.

14.4 Refuses

A plan with no criteria: the verifier would have nothing to judge, and the only grader left would be the agent that did the work. A plan silently repaired to look complete — problems are reported, not patched, because a plan quietly fixed is one the owner reads as authored.

14.5 Characteristic failure

A draft that improves in wording and not in substance across rounds, consuming the round budget. The fingerprint on the draft makes “did this actually change” answerable rather than a matter of impression.

14.6 Deployment status

Built, including the workspace seeding (§5) and the joint rounds. This is the newest plane in the system.

15 Session — `session.py` (397 lines)

15.1 Role

The worker hands over the plan and keeps the session. Every control — steer, answer, stop, resume, turn off, governor, engine — is mediated here.

15.2 Two rules, enforced rather than described

Rule 13 (The worker is handed the plan, not the wish). *The session receives the plan file — phases with their criteria — because the verifier judges those criteria. Handing over the owner’s one-line request instead would leave nothing to judge against, and the verdict would be the session’s opinion of its own work.*

Rule 14 (A session belongs to the worker that started it, for that session’s whole life). *Every control checks ownership first. The machine agent raises on all of them: it plans, routes and answers, and does not execute. Another worker raises too.*

15.3 Interface

Session controls — every one checks ownership first

`assign` — hands over the plan and starts the session. Raises for the manager, and raises without a plan.
`steer(name, text) · answer(name, text)` — both send; they differ in what the record says about who decided.
`stop · resume · turn_off · governor(name, ceiling)` — the owner’s controls, mediated so that a non-owning agent cannot reach them.
 A backend abstraction stands between these and the runtime, so an absent runtime is a distinguished state rather than a crash.

15.4 Refuses

Starting without a plan. A control from a non-owning agent. Every control from the manager.

15.5 Characteristic failure

Ownership drift over time as controls are added — a new control written without the check. The remedy in place is that the check is a single call every control makes as its first line, so its absence is visible in review.

15.6 Deployment status

Built, with a backend abstraction so the runtime can be absent in tests.

16 Answering — `answer.py` (165 lines)

16.1 The two stops

A session halts for two reasons, and on a terminal they are the same event: text on a screen, waiting.

- “Which directory should I index?” — a request for a **fact**, very likely one the worker already holds, written in the plan it seeded.
- “Do you want to proceed?” — a request for **authority**, asking whether the session may act.

The failure this plane exists to prevent

Conflating the two is how an agent approves its own actions because it “knew what the owner would want.” Each step of that reasoning is locally plausible, and the result is an agent spending authority nobody gave it.

16.2 The order is the mechanism

Rule 15 (The permission check runs first, and the model is never shown a gate). *Gate detection precedes everything, and a detected gate is declined outright. The question is not passed to a language model, because a model shown a gate may reason its way to an answer. Gates stay on their existing path: the ceiling, a standing grant the plan anticipated, or the owner.*

The distinction between *instructing a model to decline* and *never showing it the question* is the whole point. The first leaves the property to the model’s judgement on the day; the second makes it a property of the control flow.

Rule 16 (Phrasing does not launder a reserved class). *A question naming money, consent, publishing or legal goes to the owner regardless of grammatical form. “Which card should I pay the invoice with?” is a money question wearing a choice’s clothes, and a detector tuned to “do you want to” will not catch it.*

Rule 17 (An answer is grounded or refused). *The worker answers only from the plan, the directive and its own workspace notes, and otherwise returns an explicit I do not know and asks the owner — once, keyed, closing from either surface.*

A guessed answer sends a session confidently in the wrong direction, and does so in the owner’s voice. The decline is therefore a first-class outcome rather than an error path.

16.3 Deployment status

Built, including the reserved-class check and the rung recording (Rule 3).

17 Vault — `vault.py` (197 lines)

17.1 Role

Policy first, then ask, and the secret never leaves.

An agent never receives a credential

It asks the vault to *use* one. What crosses is a requirement and an answer; the value stays where it is. The consequence worth stating: a component that can plan work requiring a credential need never be able to learn it.

17.2 Two stages

Standing policy answers what it matches, without interrupting. Everything else asks the owner for this one use. Promotion from the second to the first is an owner act: an agent that earned a standing grant by being approved repeatedly would have acquired authority by persistence, invisibly, because the fifth approval looks like the first four.

17.3 Refuses

Emitting a secret into a directive, a report, a plan, a prompt, or a log. Letting any agent enumerate the store — the only interface is the question.

17.4 Deployment status

Built, with a real transport through it.

18 Outward Acts — `outward.py` (218 lines)

18.1 Role

The gate every act that reaches a person passes through.

Rule 18 (Drafting is not sending). *An agent may compose anything it likes. Composing is not sending, and the act of transmission requires a grant naming that act.*

Rule 19 (Approved bytes are the transmitted bytes). *What the owner approved and what goes out are identical. No reformatting, truncation or template substitution happens in between.*

The second rule closes a gap the first leaves open: an approval workflow that renders the message differently at send time has obtained consent for something other than what it did.

18.2 Deployment status

Built.

19 Verification — `verify.py` (192 lines)

19.1 Role

A separate judge, given the plan’s criteria and the session’s visible evidence, rules on whether the work meets the standard.

Rule 20 (The session does not decide whether it succeeded). *The judge is independent of the agent that produced the work, and it judges the plan’s criteria rather than the agent’s account of itself.*

19.2 Refuses

A verdict without criteria. A verdict from the working agent. A claim of success unsupported by visible evidence.

19.3 Characteristic failure

Criteria vague enough to be satisfied by any outcome — which is why the planning plane refuses a plan without them and the joint rounds check them against what the verifier will need.

19.4 Deployment status

Built, with an unavailable-judge path so the absence of a model is distinguished from a failed verdict.

20 Modes — `modes.py` (216 lines)

20.1 The three modes

Table 4: Guided, Interact, History.

Mode	Behaviour
Guided	the owner steers <i>through the worker</i> , so their guidance lands under the same ownership check as the worker’s own. Nothing here reaches a session directly, which is what keeps the manager out of it
Interact	the owner takes the wheel: steering pauses, the plan is marked stale, and the mode <i>reminds</i> the owner to attach to the session rather than relaying keystrokes into it
History	what was said, on either surface

20.2 Precedence is mechanical

Rule 21 (Interact pauses the worker’s steering). *Otherwise the worker keeps typing its next step into a terminal the owner is using, and the two contend for the prompt.*

This is how *the owner outranks the worker* becomes a property rather than a convention. A system in which precedence is agreed but not implemented has precedence exactly until two things want the prompt at once, which is the only moment it matters.

The plan is marked stale in the same operation, for a separate reason: the owner is about to do something the plan does not describe, and a plan surviving that would send the worker back through phases the owner has just overridden by hand.

20.3 Deployment status

Built. Interact deliberately does *not* relay keystrokes — it points the owner at the real session, because a second relay into one terminal is the contention the pause exists to prevent.

21 Board and Commands — `board.py` (116), `commands.py` (162 lines)

21.1 Role

The same task list on every surface: the command line, the chat, and the console page. Three faces, one source.

21.2 Refuses

Divergence between surfaces. A board assembled independently per surface is three boards that agree until they do not.

21.3 What it shows

Every task with its *reason* — not merely its state. A state without a reason makes the owner ask a follow-up question the system already knows the answer to.

21.4 Deployment status

Built on all three surfaces.

PART III — SURFACES AND CONSEQUENCES

22 The Console Above It

The console requires the agent runtime, installs it when absent, and opens gates the runtime keeps shut standalone so that agents may reach the manager and share a workspace. The dependency runs one way; the runtime never depends on the console, which is what gives the gates meaning — a gate signifies nothing unless some state has it shut.

Design principle 5 (A missing prerequisite is a fault, not a state). *Code that catches the absence and continues with an empty result hides a broken installation behind an empty page: the operator is shown no agents and given no reason.*

The console’s surface over this system is deliberately read-only — three routes and a page, exposing agents, their tasks, and one task’s plan with its criteria and declared permissions. Two absences on that page are the design, and both are asserted by tests:

- **no progress bar**, because per-phase completion is recorded nowhere and a filled bar would be a number the page invented;
- **a plan that cannot be parsed says so**, because an empty phase list would read identically to “*no plan yet*” — a different and ordinary state.

Editing from the web is deliberately absent: it is a mutation, and it belongs with an authority story that has not yet been written.

23 A Request, Traced

Specifications describe planes. This section follows one request through all of them, because the interactions are where the design is either coherent or merely tidy.

23.1 The request

The owner sends, from a phone, over the chat surface:

Inbound

“regenerate the CASSI figures for the grant — the colour bar was wrong in figure 4”

23.2 RT — routing

The bindings table maps this (channel, account) pair to an agent. The owner lock is consulted: the sender is on the allowlist, so the message proceeds. Had it not been, the message would have been dropped and counted, with no reply and no transcription — the sender would learn nothing, including whether anything is listening.

No worker is named in the text, so the manager will route it to the base worker rather than infer a specialist from the word “figures” (Principle 1).

23.3 tsk — a task, not a session

A task is created in state *drafting*. **Nothing has started.** This is Rule 1 in operation: the owner has asked for something, and has not yet authorised work.

23.4 PLN — the plan is made between two parties

The worker reads its own workspace first: notes it wrote, the conversation across both surfaces, what it learned from prior tasks. Suppose that workspace contains a correction the owner made three weeks ago — *the figures live in the grant folder, not the manuscript folder*. The seed plan therefore names the right path, which a fresh model drafting from the request alone could not have known.

The seed is written to the draft file. A planning session opens on that path, capped below the work ceiling and told to plan and not to do. It reads the actual repository — and discovers that figure 4’s generator takes a mask argument the plan did not mention. It improves the draft in place.

The worker reads the draft back and checks it against what the verifier will need: does every phase carry a criterion? It finds phase 3 has none, and guides one more round. The second draft passes. The worker promotes it.

What each party contributed

Neither could have produced this plan alone. The worker supplied the owner’s history — the path correction. The session supplied the repository — the mask argument. The worker supplied the standard — criteria on every phase. And the session did not author its own criteria, so the verdict that eventually judges this work will mean something.

23.5 GRT — the permissions the plan declared

The plan declares writing to the grant folder and running the numerical toolchain. Neither is granted, so the task moves to *awaiting-grant* — a state that says what is true, rather than sitting in *ready* and failing later.

The owner grants both from the phone. Asking here is the entire point: the worst moment to request permission is halfway through unattended work.

23.6 ASG — the owner opts in

The task is *ready*, and it stays there until the owner says start. When they do, the worker hands over **the plan** — not the original sentence — and keeps the session. The manager could not have reached this step; the call raises for it.

23.7 CC — and a question

The session works, and stops with:

On screen

“Which mask should figure 4 use — the continuous one or the binary one?”

The answering plane runs its checks in order. It is not a permission gate, so the model is not declined on that ground. It names no reserved class. Can it be answered from the plan, the directive, or the workspace? The plan — improved by the session itself two steps ago — specifies the continuous mask. The worker answers, and records the answer at the inference rung, not the owner’s.

Later the session stops again:

On screen

“Do you want to make this edit to `cassi_models.py`?”

This one is a gate. It is declined *before any model sees it* and routed to the ceiling, the standing grant, or the owner. The distinction between the two stops is invisible on the terminal and total in the system.

23.8 VER — and REP

An independent judge is given the plan’s criteria and the session’s visible evidence. Not the worker’s account, and not the session’s. It rules.

The report returns up the same conversation the owner started on their phone — and had they meanwhile opened the command line, they would see the same board, because there is one conversation per agent and not one per door.

23.9 Where the owner was involved

Four times: the initial request, the grant, the start, and the gate. Not once for the mask question, the path, the criteria, or the toolchain — and each of those is a place where a system without a workspace, without split plan authorship, or without the answering order would have interrupted them.

24 Below ASG: The Session Plane

This paper stops at ASG because what lies below it is specified separately: a twenty-seven-node loop that drives one session from a registered goal to a verified result. Three of its properties matter here, because they are what the planes above rely on.

24.1 Verification is an ordering rule, not a step

Within each supervision pass, the loop asks whether the goal is already met *before* it asks what to type next. The position is load-bearing rather than aesthetic: a session that kept receiving typed prompts once consumed every pass at the earlier step, so verification never ran — it starved for twenty-three consecutive passes while an already-finished session went on being guided.

24.2 Reporting is not ruling

The loop scans each round’s output for failure signatures — tracebacks, failing tests, build errors, unmerged conflicts — with a literal string match and no model call. It reports; it never rules.

Design principle 6 (Silence is not success). *A scan that matches nothing means no signature matched, not the work is good. Most ways of being wrong leave no recognisable trace. The renderer returns an empty string rather than a reassurance, precisely so nothing downstream can mistake quiet for correct.*

24.3 The stuck detector separates four situations

Four passes without transcript growth is one signal covering several conditions: an unreadable permission store, an exhausted context, a superseded plan, or a genuine wedge. The loop separates them, and only the last reaches the owner — the others abstain, recover, or delegate. This is why the planes above see relatively few escalations: most have already been resolved below.

25 The Seam to the Console, in Full

The console reaches the agent planes through exactly one module, and the shape of that module is a design decision worth stating.

25.1 One module, plain data

The seam

```
list_agents() -> [{name, kind, available}]
list_tasks(agent) -> [{id, goal, state, plan_version, phase_total, stale,
awaiting}]
get_task(agent, id) -> {..., phases: [{n, title, verified_when}],
permissions: [...]}
```

Three properties are deliberate. It is the *only* module that knows the agent planes exist, so the coupling has one location a reviewer can hold in context. It returns **plain dictionaries**, so no agent-plane type crosses into the console and a dataclass change on one side cannot reach the other. And the imports are performed inside functions rather than at module scope — not to make the dependency optional, but so the seam is testable on a machine where the agent planes are absent.

25.2 Why the surface is read-only

Three routes and a page, and no verb that writes. This is not caution for its own sake: editing a task from the web is a mutation, and a mutation needs an answer to *who is asking* that the console’s session gate does not supply on its own. Shipping the read half first puts the information in front of the owner without deciding the authority question badly under time pressure.

25.3 Two absences that are the design

Rule 22 (No progress bar). *Per-phase completion is recorded nowhere. A filled bar would be a number the page invented, and a plausible one — which is worse than an obviously missing one.*

The page shows a phase *count* and the task’s state, and says why the bar is absent. Both are asserted by tests, so restoring the bar requires deleting a test that explains itself.

Rule 23 (A plan that cannot be parsed says so). *An empty phase list is rendered differently from a plan that failed to parse, because otherwise “this plan is malformed” and “no plan yet” — an ordinary early state — appear identical.*

25.4 The failure this surface had to survive

A single malformed plan file once emptied an entire agent’s task list: the parse raised, the listing call propagated it, and every task vanished from a page whose only job is to show tasks. One bad file is *data*, not a broken installation, so the guard is per task — a task whose plan will not parse still appears, with its goal and state, marked unreadable.

The distinction that governs the whole seam is the same one: a missing dependency is a fault and must surface; a malformed record is data and must be contained.

26 Failure Modes This System Adds

27 A Second Trace: When It Reaches the Owner

The first trace (§23) followed a request that mostly succeeded. This one follows the paths that do not, because a system is characterised by what it does when it cannot proceed.

Table 5: What can go wrong that could not before, and what catches it.

Failure	What it looks like	What catches it
Self-approval	the agent answers its own gate	Rule 15 — the model never sees it
Laundered class	a money question phrased as a choice	Rule 16
Confident guess	a wrong fact sent in the owner’s voice	Rule 17
Circular criteria	the session writes what it is judged by	§14’s split authorship
Silent queue	unplaceable work that looks in progress	Rule 8
Prompt contention	worker and owner type at once	Rule 21
Stranger’s foothold	refused text written into the record	Rule 5
Authority by persistence	repeated approvals become standing	§17’s owner-only promotion
Consent for other bytes	approval rendered differently at send	Rule 19
Flattering state	a blocked task shown as running	Principle 4

27.1 No machine can do it

The owner asks for work requiring a tool this host does not have. The worker’s capability check answers before anything is queued, and the reply names what is missing and where it would have to be.

What it must not do is wait. An unplaceable directive that sits in a queue is indistinguishable, from the chat, from work in progress — and the owner learns nothing until they ask. Rule 8 makes the refusal immediate and specific.

27.2 The plan declares a permission that is never granted

The task sits in *awaiting-grant*. It does not start, and it does not fail. It also does not sit in *ready*, which would be a small lie with a large consequence: the owner would see a task apparently about to begin, and eventually a failure that looked like a defect rather than a missing decision.

27.3 The vault denies

A step needs a credential. Policy does not match, so the owner is asked — for this one use, naming the secret and the purpose. They decline.

The step fails, the task records why, and **nothing about the denial teaches the system to stop asking**. A denial is not a policy; converting one into standing policy would be the system writing rules from the owner’s refusals, which is the mirror image of the authority-by-persistence failure the promotion rule prevents.

27.4 The session asks something nobody can ground

The session asks a question the plan does not answer, the directive does not cover, and the workspace has no note about. The worker returns its explicit decline rather than a plausible guess, and the question goes to the owner — once, keyed, closing from whichever surface answers it.

Why the decline is a first-class outcome

The tempting alternative is a best-effort answer with a hedge. It is worse than it looks: the hedge is visible to the owner, and invisible to the session, which receives a confident instruction and acts on it. The asymmetry is the argument — a caveat helps only the party that is not going to act.

27.5 The verifier fails the work

The judge is given the plan’s criteria and the visible evidence, and rules against. The verdict is recorded with its reason, and the reason is used rather than merely logged: it is what the next round of guidance addresses.

A failed verdict is not a terminal state. What is terminal is a failed verdict the loop cannot act on — which is why the criteria are checked for judgeability during planning, before any work is authorised.

27.6 The owner takes the wheel mid-flight

They enter interact mode. Steering pauses, the plan is marked stale, and the mode points them at the real session rather than relaying keystrokes.

Each of the three does something distinct. Without the pause, the worker types over them. Without the staleness mark, the worker resumes afterwards through phases the owner has just overridden by hand. Without the pointer — if the console relayed instead — two paths would write to one terminal, which is the contention the pause exists to prevent, reintroduced by the interface.

28 A Taxonomy of the Refusals

Twenty refusals are catalogued in Appendix A. They fall into four kinds, and the kinds turn out to be more useful than the list, because each kind fails differently when it is absent.

28.1 Authority refusals

The manager executing; a non-owning agent controlling a session; an agent answering its own gate; a reserved class answered by inference; a standing grant earned by repetition.

These protect the boundary between what a system may do and what a person must decide. Their common failure mode is **gradual and locally reasonable**: each individual crossing has a good argument, and the aggregate is a system that acts on its own authority. They are the refusals that must raise rather than warn, because a warning is an argument’s opening move.

28.2 Evidence refusals

A verdict from the working agent; a plan whose criteria the session wrote; success claimed without a verdict; an answer without its rung.

These protect the meaning of the system’s own claims. Their failure mode is **silent and total**: nothing breaks, outputs keep arriving, and every one of them is worth less than it appears. A system that has lost these still reports success at the same rate — which is precisely why it cannot detect the loss.

28.3 Honesty refusals

A blocked task shown as running; unplaceable work waiting quietly; silence read as success; a plan silently repaired; an empty phase list where a plan failed to parse.

These protect the owner’s model of what is happening. Their failure mode is **delayed**: the system is correct and the owner is misinformed, and the discrepancy surfaces only when they act on the belief. Each of these was implemented as a distinct state or an explicit message rather than as an absence, because absence is what the owner fills in with an assumption.

28.4 Blast-radius refusals

A secret leaving the machine; a stranger’s text entering the record; sending bytes the owner did not approve; a test process reaching the live host.

These bound what a failure can reach. Their failure mode is **rare and severe** — the inverse profile of the other three, and the reason they are enforced structurally rather than by policy. The vault does not decide to keep a secret local; it has no interface that would let it leave.

Design principle 7 (The four kinds need different enforcement). *An authority refusal must raise. An evidence refusal must be structural, since its absence is undetectable from output. An honesty refusal must be a distinct state, since a missing signal is read as a benign one. A blast-radius refusal must be an absent capability rather than an unexercised one.*

29 How This Was Verified

A report claiming a system holds certain properties should say how that was checked. The suite is 933 tests over the console and the agent planes. Three things it caught are worth recording, because each is a class of defect rather than an incident.

29.1 Tests that passed for the wrong reason

Three appeared in a single subsystem, and all three would have survived any amount of green output.

Table 6: Green, and not testing what it claimed.

The test	Why it proved nothing
A withheld-criteria test	It set <i>both</i> flags that trigger withholding. Had one been dropped from the implementation, the test would still have passed — so it could not detect the loss of the protection it existed for
A registry-listing test	Its fake object carried a field the real one does not have, so the code path under test was the fallback, never the real one. The asserted value was never checked at all
A phase-count test	The function’s body ran in <i>zero</i> tests: the only case that reached it replaced it with a stub. Both its branches were unverified, and it produces the number the interface rests on

Design principle 8 (A suite is evidence about the tests, not about the code). *Each of these was green from the day it was written. What found them was a reviewer asked to check whether a test isolates the condition it claims to guard — a question no test runner asks.*

29.2 Tests that touched the live host

Two defects had the suite reaching out of its process into the running machine. In the first, a background thread spawned during one test woke thirty seconds later — inside an unrelated test — and swept the host’s

real session fleet, writing records into whichever temporary directory that later test had configured. It failed a deployment gate at random and passed in isolation every time.

In the second, a shared fixture ran the application’s startup events, and one of those posted the machine’s real credential and a fleet summary to a live endpoint. Its own docstring asserted that it was safe.

Design principle 9 (The suite is part of the system’s blast radius). *Both defects were introduced by attempts to make tests more realistic, and neither is visible in a passing run. A test process that can reach the live host is a component with production access and no review.*

29.3 What the source corrected in the design

Seven assumptions written into an implementation plan were wrong about the code they described — field names, an interface that could not exist under the plan’s own constraints, an authentication step omitted from a sample. Every one was caught by an implementer or reviewer reading the source rather than the plan.

Design principle 10 (The source outranks the document that describes it). *Stated in every task instruction, this single rule accounted for all seven corrections. The documents were accurate about intent and wrong about detail, and detail is what runs.*

30 Two Sessions, One System

This system was built by two agentic development sessions working concurrently in one repository, and that is worth reporting because the failure modes were not the expected ones.

30.1 What did not go wrong

Merge conflicts were absent. Twelve commits landed on the shared branch from one session while the other developed ten in an isolated worktree, touching a disjoint file set, and the merge was clean on the first attempt.

30.2 What did

Duplicated design, not duplicated text

Three components were designed twice, independently, in nearly the same terms: the answering plane, the planner’s workspace, and the joint planning loop. In each case the second session’s design document was written while the first session’s implementation was already merging.

The duplication was invisible to both — version control shows a conflict when two parties edit one file, and shows nothing at all when two parties solve one problem. The cost was not lost work but re-derived work, and it was caught only by inventory: reading the tree before planning, rather than planning from a prior document.

A second, subtler cost: an isolated worktree branched before the other session’s package existed, so a module had to be brought across to make the seam importable — creating for a time two copies of one file with no mechanism to notice if they diverged.

Design principle 11 (Concurrency in agentic development is a coordination problem, not a merge problem). *The tooling addresses the case it can see. Two agents solving the same problem in different files produce no signal at all, and the only reliable detector found here was to inventory the tree immediately before planning — treating one’s own prior design document as the least trustworthy input.*

31 What Is Not Built

1. **Settlement** — the usage fee, the exchange node, the initial non-exchangeable balance. Designed; absent.
2. **The manager does not front the console.** It exists and routes; it is not yet the way in.
3. **Multi-machine.** Everything here is one host. The fleet form is designed [1] and unbuilt, and §5's workspace is per agent rather than per agent per host.
4. **L4 is thin.** Outcomes are recorded and not yet aggregated into anything that reads them back, so competence is not measured from them.
5. **Owner precedence during joint planning is unspecified.** The steering pause covers interact mode; what happens when the owner guides while worker and session are converging on a draft is not settled.
6. **Per-phase completion is recorded nowhere**, which is why the console shows a phase count rather than progress.

32 What the Owner Is Asked, and How Often

The purpose of nearly every mechanism in this report is to make an interruption unnecessary without making a decision the owner should have made. It is therefore worth stating exactly where the owner is reached, and what each of those interruptions costs them.

32.1 The complete list

Table 7: Every path that reaches the owner, and what it asks of them.

Interruption	Asks for	Could it have been avoided?
The grant	permission the plan declared	No. This is the interruption the plan process exists to <i>concentrate</i> : asked once, before work, rather than repeatedly during it
The start	authorisation to begin	No, by Rule 1. Removing it would merge asking with authorising
A permission gate	whether the session may act	Only by a standing grant the plan anticipated. Never by inference
A reserved class	a decision the owner keeps	No, at any confidence
An ungrounded question	a fact nothing holds	Yes — and usually by the workspace, which is why an agent without history interrupts more
A vault use with no policy	this one use of a secret	Yes, by standing policy the owner wrote. Not by repetition
An outward act	the exact bytes leaving	No. This is the one the whole outward plane exists to guarantee reaches them
A failed verdict	judgement on what to do next	Sometimes — the loop retries first, and reaches the owner when it cannot

32.2 Which are load-bearing and which are debt

Four of the eight are structural: the grant, the start, a gate, and an outward act. Each corresponds to a decision that is the owner’s by definition, and removing any of them would not make the system more autonomous — it would make it autonomous about something that was never its to decide.

The other four are *debt*. An ungrounded question means the workspace lacked a fact it could have held. A vault prompt with no matching policy means the owner has not been given a way to express a rule they would obviously write. A failed verdict reaching the owner means the loop could not act on its own diagnosis. Each is an interruption the system caused rather than one the problem required.

Design principle 12 (Count interruptions by kind, not by number). *A system that reduced its interruption count by answering more gates would be worse, not better. The number worth tracking is interruptions of the second kind — the ones a better-informed system would not have needed — and that number is only meaningful when the first kind is fixed by construction.*

32.3 Why this framing matters for autonomy claims

Autonomy measured as *fewer stops* rewards exactly the wrong change. The agent that answers its own gates stops less than the one that does not, and it is not more capable; it has moved a decision from the owner to itself and reported the theft as progress.

This is why the answering plane records the rung of every answer (Rule 3), and why abstaining on a reserved class is not counted as a failure to be autonomous. An act no permission could authorise was never autonomously completable, so a system that abstains on it has not fallen short — it has correctly identified something outside its remit.

33 Limitations, and What This Report Cannot Claim

An implementation report is a claim about a system at a moment, made by the parties who built it. Its weaknesses should be stated by the same document.

33.1 Refusals are verified structurally, not adversarially

Each refusal in Appendix A has an enforcement point and a test. What has *not* happened is an adversarial attempt to defeat them — no one has tried to phrase a permission question so the gate detector misses it, or to construct a plan that passes the criteria check while remaining unjudgeable. The reserved-class check in particular matches on vocabulary, and vocabulary is exactly what an adversary varies.

The honest status of the answering plane

It is correct on the cases it was built for and on the cases tests describe. Its detection is a string match, and every string match has a complement. The claim “an agent cannot answer its own gate” is warranted for gates that look like gates; it is not a proof.

33.2 Scale is one owner, one host, and a short history

Every property here was exercised at a scale where the owner sees most of what happens. Several are load-bearing precisely when they cannot — the honesty refusals matter most when there is too much activity to follow, and that condition has not been reached. Concurrency limits, ledger growth, and workspace size are untested at volume.

33.3 The self-model is thinner than its papers

The architecture specifies ten self-state coordinates with per-coordinate evidence and update rules. What is built records outcomes without aggregating them into a competence estimate, so *measured* capability does not yet feed any decision. Where a paper says an agent knows what it can reliably do, this system knows what it did.

33.4 Two of the four channels are absent

Report and directive are built; prior bias and owner pre-emption as a distinct channel are not. Pre-emption exists in effect — the steering pause — but not as the general mechanism the design describes, so its properties hold for interact mode and are unproven elsewhere.

33.5 The report was written by a participant

This document was produced by one of the two agentic sessions that built the system, which is a conflict of the exact kind the architecture forbids elsewhere: the party that did the work should not grade it. Read the deployment statuses and the failure inventory with that in mind. The mitigations are that every claim was checked against source rather than against prior documents, and that §29 reports defects found in this session’s own work — including a test fixture whose docstring asserted a safety property it did not have.

34 Conclusion

The design papers in this series argue for properties. This one reports which properties survived being built, and the answer is consistent enough to state as a rule: **the ones implemented as refusals at the point of action, rather than as rules stated in one place and honoured in another.**

The manager cannot drive a session because the call raises. Owner precedence holds because interact mode pauses the worker. An agent cannot answer its own gate because the model is never shown one — not because it is instructed to decline, which would leave the property to the model’s judgement on the day. A plan cannot be circular because its authorship is split between two parties with different interests, through a file that can be diffed.

The counter-examples are equally consistent. Every property that decayed during implementation was one recorded as a description: the planner’s context parameter that nothing filled, the workspace sources declared and never fed, a fixture docstring asserting a safety it did not have. In each case the documentation was accurate about the intent and silent about the wiring, and the wiring is what runs.

Everything in such a system that is merely written down remains a hope. The work of building it was, in almost every case, the work of finding the place where a hope could be made into a refusal.

A Index of Refusals

Every refusal in the built system, with where it is enforced.

Refusal	Enforced at
A stranger is answered	the router, before any handler
Refused text is recorded	the router — counted, not transcribed
The manager executes	the assignment call raises

Refusal	Enforced at
A session starts without a plan	the assignment call
A non-owning agent controls a session	every control’s first line
The machine agent executes	every control
A plan without criteria	the planner’s check, before the draft is returned
A plan silently repaired	the planner reports problems instead
The session authors its own criteria	split authorship, worker promotes
A gate is shown to a model	the answering order, first check
A reserved class is agent-answered	the reserved-class check, any phrasing
A guessed answer	the explicit decline path
An answer without its rung	recorded on every outcome
A secret leaves the machine	the vault’s interface: a question, not a read
A standing grant earned by repetition	promotion is an owner act
Sending what was not approved	byte equality at transmission
A verdict from the working agent	the independent judge
A blocked task shown as running	the state set
Unplaceable work waiting quietly	refused and reported together
The worker typing over the owner	the steering pause

B Deployment Status by Plane

References

- [1] Chengshuai Yang. One account, many machines: Designing the multi-host SARSI console — per-host agents, host-indexed evidence, channels that can drop, and what a machine takes with it when it leaves. Preprint. The immediate predecessor of this paper, 2026.
- [2] Chengshuai Yang. One chat box, five agents: Designing the manager-fronted SARSI console — per-agent self-awareness, nested workspaces, and the four channels by which agents affect each other. Preprint, 2026.
- [3] Chengshuai Yang. Functional self-awareness for SARSI agents: Version 2.0 (history-aware revision) — selective episodic encoding, replay, consolidation, and bounded retrieval. Preprint, 2026.
- [4] Chengshuai Yang. A manager is not a controller: Global-workspace coordination for SARSI fleets. Companion paper on coordination semantics, 2026.

Table 9: Nineteen planes; all built, with the noted limits.

Plane	Lines	Status
Registry	262	built; not-initialised distinguished from broken
Router	126	built, both surfaces
Gateway	113	built; transport injected for test
Conversation	123	built; keyed questions close from either door
Manager loop	139	built
Worker plane	191	built
Tasks	343	built
Planner	128	built; workspace-seeded
Joint planning	378	built; newest plane
Session	397	built; backend abstracted
Answering	165	built; rung recorded
Vault	197	built; real transport
Outward	218	built
Verify	192	built; unavailable judge distinguished
Modes	216	built
Board	116	built, three surfaces
Commands	162	built
Workspace	105	built; per agent, not per host
Ledgers	67	built; four files

- [5] Chengshuai Yang. Functional self-awareness for SARSI agents: Version 2.0 (corrected and expanded). Preprint. The single-agent architecture, 2026.
- [6] Chengshuai Yang. One user, several machines: Designing the SARSI worker layer — server-side communicators without execution authority, five work agents, and a vault that only ever answers yes or no. Preprint, 2026.